

LSTM Model Evaluation Report

May 15th, 2026

Abstract

This report presents a systematic evaluation of Long Short-Term Memory (LSTM) neural network architectures for multi-step CPU and memory resource forecasting within a Google Cluster Trace environment. The study examines five experimental configurations across two primary forecasting strategies — tenant-aware generalization and workload behavior learning — assessing their suitability for proactive scheduling and SLA-aware resource allocation. The dataset used is the Google Cluster Trace v3 (Cell A, Week 1, May 2019), comprising over one million observation windows at 5-minute granularity across 794 unique instances. An Exploratory Data Analysis (EDA) was conducted to characterize dataset quality, workload composition, and instance lifetime distributions, confirming readiness for LSTM ingestion. Model performance is evaluated using MAE, RMSE, SMAPE, and R^2 metrics, and results are benchmarked against recent related literature. Findings demonstrate that workload behavior learning with dedicated single-output LSTM models for CPU and memory achieves superior forecasting accuracy, with R^2 values reaching 0.968 for CPU and 0.947 for memory at a 30-minute horizon. The selected final model configuration integrates 12-hour lookback windows, 5-minute granularity, and multi-step outputs transformed into optimizer-ready scheduling parameters.

Table of Contents

Dataset – EDA Summary.....	1
Overall Models Comparison	6
Pipeline Comparison.....	7
Comparison with Research Papers.....	8
Final Model Selection	9

Dataset – EDA Summary

Source: Google Cluster Trace v3 · Cell A · May 2019 · Week 1

Key Findings

Dataset Scale & Cleanliness: Zero duplicates, and only 0.12% missing values — fully ready for LSTM ingestion.

Workload Composition: Priority Tiers — dominated by Production (53.8%) and Free (45.6%) tiers; latency-sensitive jobs are a critical 6.9% minority. Terminal Type — dominated by Kill (97.1%) and zero Finish. Scheduling Class — latency-sensitive jobs represent 6.9% of rows.

LSTM Readiness: All 794 instances exceed minimum sequence length; 97.3% of sub-sequences usable after gap detection (~1M training pairs). Per-sequence normalisation is essential due to skew and distribution shift.

1. Dataset at a Glance

Property	Value
Total rows	1,018,962
Total columns	24
Unique instances (collection_id, instance_index)	794
Unique jobs (collection_id)	49
Unique tenants (user)	32
Average windows per instance	~1,283
Observation window	7 days (Week 1, May 2019)
Measurement granularity	5 minutes (300 s)
Sampling method	Deterministic: $\text{MOD}(\text{collection_id}, 1000) < 3$ (~0.3% of jobs)

1.1 Column Inventory

All timestamps are in microseconds from $t = 0$ (600 s before official trace start). All CPU values are in NCU (Normalised Compute Units; 1.0 = largest machine in cluster). All memory values are normalised (1.0 = largest machine memory observed).

Category	Columns
Identifiers	collection_id, instance_index, machine_id, user
Timestamps	start_time, end_time, scheduled_time, terminal_time
CPU metrics	avg_cpu, max_cpu, p90_cpu, p95_cpu, req_cpu
Memory metrics	avg_mem, max_mem, req_memory, assigned_memory, page_cache_memory, mem_overcommit_ratio
Scheduling metadata	collection_type, scheduling_class, priority, priority_tier
Outcome	terminal_type

2. Data Quality

2.1 Missing Values

The dataset is exceptionally clean. Out of 24 columns, only 2 have any missing values.

Column	Missing rows	Missing %	Expected?
p90_cpu	1,240	0.12%	Yes — windows too short for percentile
p95_cpu	1,240	0.12%	Yes — same reason
All other columns	0	0.00%	—

Notable: terminal_time, terminal_type, page_cache_memory, and mem_overcommit_ratio were expected to have NULLs per the documentation but have none. This is because every instance in the sample terminated after the week ended.

Action: Forward-fill p90_cpu and p95_cpu within each sub-sequence before LSTM training. Do not drop rows.

2.2 Duplicates

Zero exact duplicate rows and zero primary key duplicates — the extraction query produced a perfectly clean, deduplicated dataset.

Check	Results
Exact duplicate rows	0
Primary key (collection_id, instance_index, start_time) duplicates	0

3. Timestamps and Instance Lifetimes

3.1 Measurement Window Duration

98.19% of windows are exactly 300 s. The 1.81% shorter windows occur when an instance starts or stops partway through a window. The consistent cadence makes this dataset well-suited for fixed-step LSTM modelling.

Metric	Value
Mean window duration	296.9 s
Windows exactly 300 s	1,000,546 (98.19%)
Windows < 300 s	18,416 (1.81%) — expected for start/stop events
Windows > 300 s	0

3.2 Instance Lifetimes

Metric	Value
Minimum	969 min (~16 hours)
25th percentile	4,437 min (~3.1 days)
Median	12,749 min (~8.9 days)
Mean	13,847 min (~9.6 days)
Maximum	39,928 min (~27.7 days)
Negative durations	0

Instance lifetimes exceed 7 days because: (a) pre-existing instances started before the trace window and (b) terminal events are recorded after the week ends. Every instance ran for at least 16 hours. The distribution is multimodal with spikes near ~3.5, ~5.9, and ~13.9 days, suggesting distinct operational groups.

3.3 Pre-existing Instances

Instance Type	Instances	% of Instances	% of Rows
Pre-existing (scheduled_time = 0)	254	32%	50.6%
Started within trace window	540	68%	49.4%

Pre-existing instances are long-running services that were already active before the trace started. CPU behaviour is identical between pre-existing and within-window instances — no need to treat them as separate populations. `scheduled_time = 0` is valid data, not missing data.

4. Population Composition

4.1 Terminal Type

All terminal events in this sample occurred after the week ended (the stability filter removed any instance that terminated within the week).

Terminal Type	Code	Instances	%
KILL	7	771	97.1%
EVICT	4	13	1.6%
FAIL / OOM	5	10	1.3%
FINISH	6	0	0%

Important: There are zero FINISH events in this sample. The Random Forest classifier designed to distinguish 'stable (FINISH)' vs 'at-risk (EVICT/FAIL/KILL)' cannot be trained on this sample alone. Full extraction or Week 2 data is required.

4.2 Priority Tiers

Tier	Priority Range	Row Count	%
Production	120–359	547,837	53.8%
Free	≤99	465,058	45.6%
Monitoring	≥360	6,067	0.6%
BEB	100–115	0	0%
Mid-tier	116–119	0	0%

The data splits almost evenly between Free and Production tiers. BEB and Mid-tier are entirely absent — a consequence of the deterministic 0.3% sample. Production instances use significantly more CPU (median ~0.008 NCU) vs Free instances (median ~0). Free-tier jobs are largely idle.

4.3 Scheduling Classes

Class	Label	Row Count	%
0	Non-production	475,146	46.6%
1	Development	328,611	32.2%
2	Batch	144,657	14.2%
3	Latency-sensitive	70,548	6.9%

Only 6.9% of rows are latency-sensitive — the class most critical for SLA compliance. Batch (class 2) has the tightest and most predictable memory profile. Latency-sensitive (class 3) has the widest memory spread, the highest median, and the most variance.

4.4 Tenant Concentration

Metric	Value
Unique tenants	32
Unique jobs	49
Average jobs per tenant	1.5
Top tenant share of jobs	18.4%
Expected top tenant share (full dataset)	~61%

The expected dominant tenant (61% of jobs in the full dataset) is not present in this 0.3% sample. This sample is not suitable for tenant-level fairness analysis. ~25 tenants are needed to cover 80% of jobs in this sample.

Overall Models Comparison

Experiment	Split Strategy	Target	MAE	RMSE	SMAPE (%)	R ²	Interpretation
Model 1	Tenant-aware split (user)	CPU	0.003189	0.007726	100.859	0.0878	CPU forecasting struggled under unseen-tenant generalization due to bursty workload behavior.
		Memory	0.000765	0.001534	37.221	0.7266	Memory forecasting remained moderately strong because memory behavior is more stable than CPU.
Model 2: 30 min horizon	Workload behavior learning	CPU	0.000803	0.001878	63.908	0.9587	Allowing workload behavior across train/test greatly improved temporal pattern learning.
		Memory	0.000091	0.000197	6.397	0.9563	Memory forecasting became highly accurate under workload behavior learning.
Model 3: 1 hr horizon	Workload behavior learning	CPU	0.000859	0.001958	71.146	0.9552	Longer-horizon CPU forecasting remained stable with only slight performance degradation.
		Memory	0.000095	0.000210	6.561	0.9499	Memory remained highly predictable even at a 1-hour forecasting horizon.
Model 4	Workload behavior learning	CPU	0.000599	0.001649	36.992	0.9682	Separating CPU into its own model improved CPU specialization and temporal workload learning.
Model 5	Workload behavior learning	Memory	0.000065	0.000232	5.598	0.9473	Dedicated memory forecasting remained highly accurate with strong temporal stability.

Pipeline Comparison

Component	Model 1	Model 2	Model 3	Model 4	Model 5
Dataset	Google Cluster Trace Week 1	Same	Same	Same	Same
Granularity	5 min	5 min	5 min	5 min	5 min
LOOKBACK	144	144	144	144	144
Horizon	6	6	12	6	6
Split Method	Tenant-aware	Chronological workload-learning	Chronological workload-learning	Chronological workload-learning	Chronological workload-learning
Same workload across train/test	No	Yes	Yes	Yes	Yes
Duplicate sequence removal	Partial	Yes	Yes	Yes	Yes
Targets	CPU + Memory	CPU + Memory	CPU + Memory	CPU only	Memory only
Architecture	Dual-output LSTM	Dual-output LSTM	Dual-output LSTM	Single-output CPU LSTM	Single-output Memory LSTM
Main Evaluation Goal	Generalization	Workload behavior learning	Horizon robustness	CPU specialization	Memory specialization

Comparison with Research Papers

Study	Dataset / Task	Metrics Used	Reported Results
Alshboul et al. (2026), "An LSTM-Based Resource Prediction Model in Google"	Google Cluster 2019, CPU & memory request prediction	MSE, MAE, RMSE	CPU: MSE ~0.00132, MAE ~0.0216, RMSE ~0.0364. Memory: MSE ~0.0029, MAE ~0.0249, RMSE ~0.0537.
Kofi (2025), "Data-Driven Cloud Workload Optimization Using Machine Learning Modeling for Proactive Resource Management"	Google Cluster Trace, workload/CPU/RAM prediction	R ² , MSE, RMSLE	CPU: R ² 0.99, MSE 128.89, RMSLE 0.16. RAM: R ² 0.99, MSE 131.29, RMSLE 0.14.
Kidane et al. (2025), "Hybrid Autoencoder–LSTM Framework"	Synthetic + Wikipedia workload forecasting	RMSE / prediction loss, computational metrics	Reports RMSE/prediction loss and horizon-based errors, but not directly CPU/memory Google Cluster values.

Alshboul et al. is the closest comparison because it uses Google Cluster data, LSTM, and CPU/memory prediction with MAE/RMSE/MSE. Their feature selection is more 'usage-history based' while ours is more 'EDA-driven + domain-aware'.

How They Predict CPU and Memory

- They create sequential workload windows from Google Cluster data.
- Past workload values are fed into the LSTM.
- The LSTM learns temporal workload patterns.
- The model predicts future CPU demand and memory demand.

Our Approach

Our approach also uses Google Cluster trace data and LSTM-based time-series forecasting for CPU and memory prediction. However, the pipeline is designed more specifically for SLA-aware workload optimization and scheduler support. Instead of relying mainly on usage-history features, the proposed framework combines EDA-driven analysis, workload behavior learning, temporal sequence engineering, and optimizer-oriented forecasting outputs. The predicted sequences are further transformed into optimization-ready parameters such as average usage, peak usage, variance, and percentile-based workload indicators to support proactive resource management and scheduling decision.

Final Model Selection

Among the evaluated architectures, **the workload behavior learning approach with separate specialized LSTM models for CPU and memory forecasting using a 30-minute prediction horizon** was selected as the final prediction framework for the proposed scheduling system.

This configuration achieved the strongest balance between forecasting accuracy, workload behavior learning, and operational suitability for proactive cluster scheduling. Compared to tenant-aware generalization, workload behavior learning is better aligned with the project objective of modeling temporal workload evolution within production cluster environments. Additionally, separating CPU and memory into dedicated models improved specialization and reduced interference between heterogeneous resource utilization patterns. The 30-minute forecasting horizon was selected over the 1-hour horizon because it provided slightly stronger predictive stability while remaining more appropriate for real-time scheduling and SLA-aware resource allocation decisions.

The final framework therefore consists of:

- Workload behavior learning split strategy
- Separate LSTM models for CPU and memory
- 12-hour lookback windows
- 5-minute workload granularity
- Multi-step forecasting outputs transformed into optimizer-ready scheduling parameters